

HW #3 Solutions

CSEE W3827 - Fundamentals of Computer Systems Spring 2026

Prof. Rubenstein

Topics: Standard circuitry (encoder, decoder mux), combinatorial circuit design

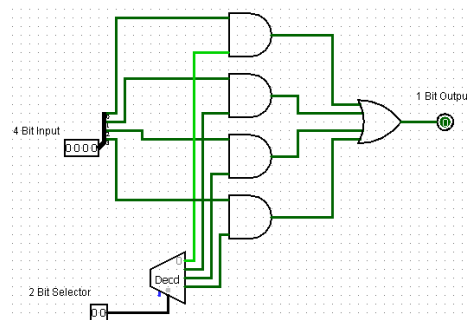
Note that this homework has 5 problems and is 13 pages long.

Warmup Problems

These problems do not need to be submitted - they are easier and can help prepare/familiarize the student with the material on the harder problems (that do need to be turned in).

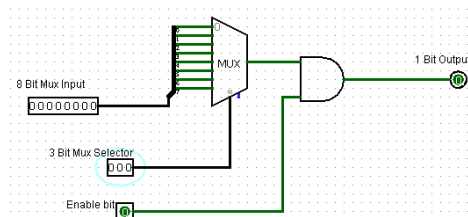
1. Build a MUX from a Decoder and some AND and OR gates

Answer:



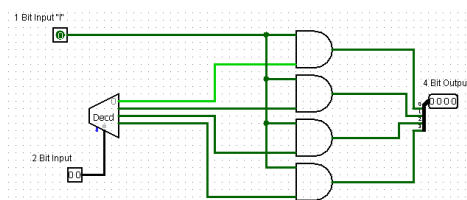
2. A MUX-with-Enable is a MUX with one additional selector input, E . When $E = 1$, the MUX-with-Enable behaves like a traditional MUX. When $E = 0$, the MUX-with-Enable is disabled and outputs 0. Build a MUX-with-Enable from a MUX (without enable) and some AND gates.

Answer:



3. A 1-to- 2^k DEMUX takes one data input I and a k -bit selector S as input and outputs 0 on each of $2^k - 1$ outputs, and outputs I on the j th output where j is the unsigned binary value represented by S . Construct a DEMUX from a decoder and a bunch of AND gates.

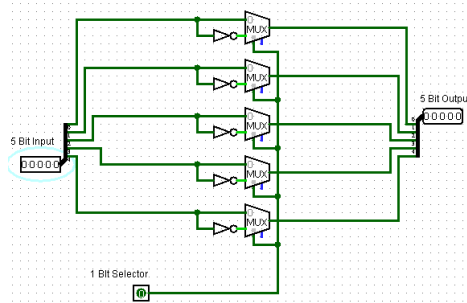
Answer:



4. Use five 2-to-1 MUXs and as many NOT gates as needed, build a circuit that takes a 5-bit value and a selector input S and returns
 - when $S = 0$, the original number is returned

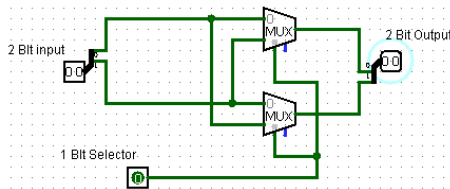
- when $S = 1$, the 1's complement of the number is returned.

Answer:



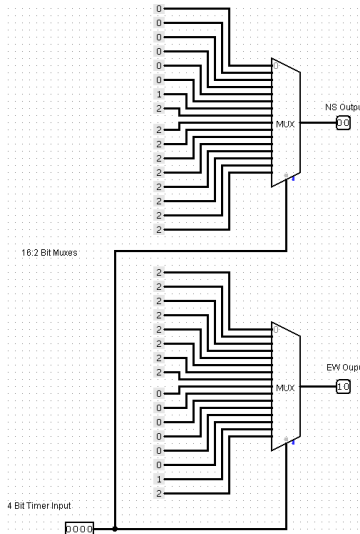
5. Build a circuit using two 2-to-1 MUXs that takes a 2 bit-input B_1B_0 and a 1-bit selector S and returns B_1B_0 when $S = 0$ and returns B_0B_1 (i.e., switches the bit-order) when $S = 1$.

Answer:



6. Solve problem 3 of the “Harder Problems” using four 16-to-1 MUXs, one MUX for each output NSH, NSL, EWH, EWL. Now solve using four 8-to-1 MUXs

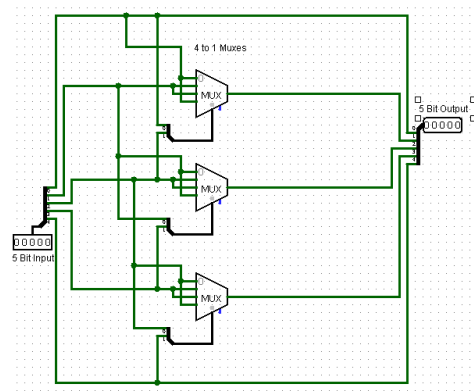
Answer:



7. Design a circuit that receives a k -bit string $A = A_{k-1}A_{k-2} \cdots A_1A_0$ and, using $k - 2$ 4-to-1 MUXs, outputs k -bit string $B = B_{k-1}B_{k-2} \cdots B_1B_0$ with the following properties

- $B_0 = A_0, B_{k-1} = A_{k-1}$
- $B_i = A_i$ whenever $A_{i+1} \neq A_{i-1}, 0 < i < k - 1$
- $B_i = A_{i-1}$ whenever $A_{i+1} = A_{i-1}, 0 < i < k - 1$

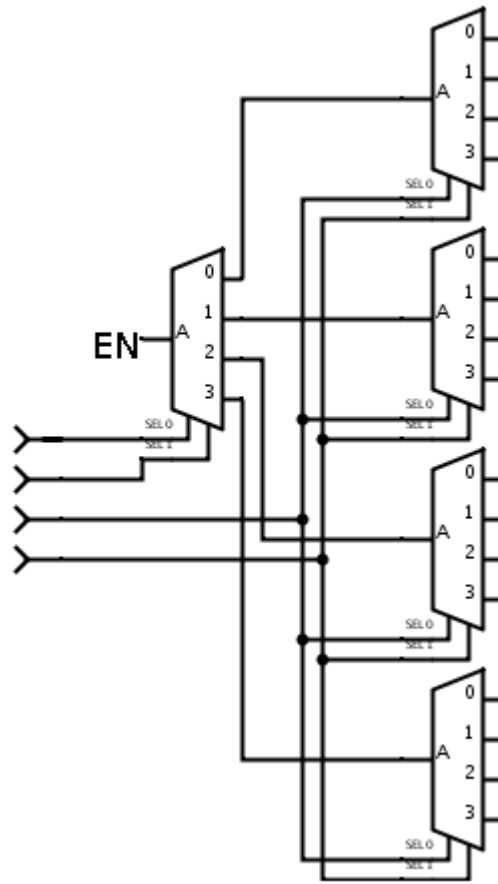
Answer:



Harder Problems

- Construct a 4-to-16 line decoder with an enable input using five 2-to-4 line decoders with enable inputs (Hint: Start at the outputs: If all that is being used is decoders, then how many decoders are connected directly to outputs?)

Answer:



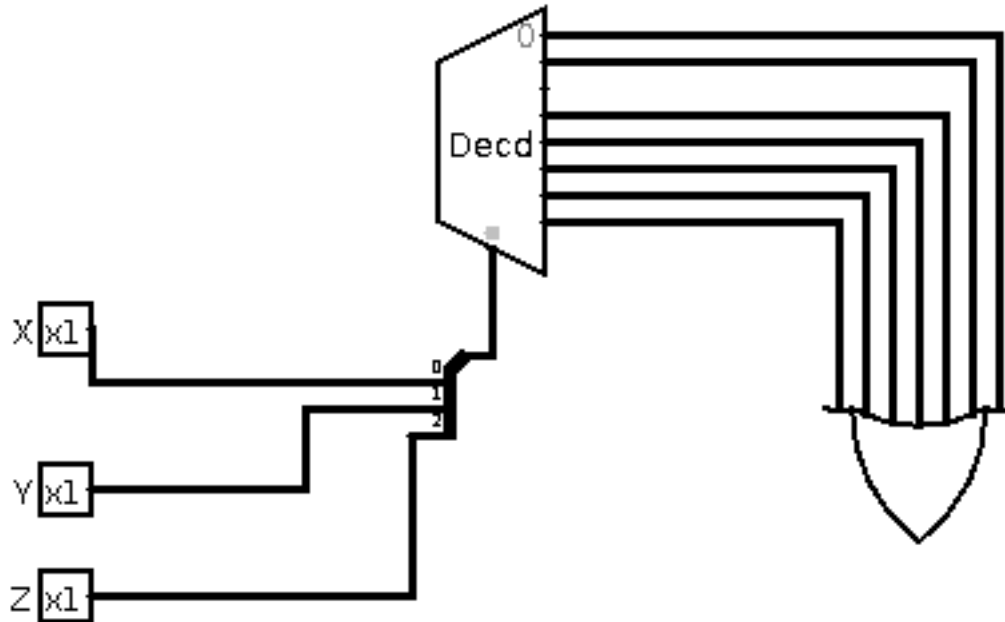
- A combinatorial circuit is specified by the following three Boolean variables:

$$F = X + \bar{Y} + \bar{X}YZ$$

Design the circuit with a decoder and external OR gates.

Answer:

$$\begin{aligned}
 F &= X + \bar{Y} + \bar{X}YZ \\
 &= (X\bar{Y}\bar{Z} + X\bar{Y}Z + XY\bar{Z} + XYZ) + (\bar{X}\bar{Y}\bar{Z} + \bar{X}\bar{Y}Z + X\bar{Y}\bar{Z} + X\bar{Y}Z) + (\bar{X}YZ) \\
 &= \bar{X}\bar{Y}\bar{Z} + \bar{X}\bar{Y}Z + \bar{X}YZ + X\bar{Y}\bar{Z} + X\bar{Y}Z + XY\bar{Z} + XYZ \\
 &= m_0 + m_1 + m_3 + m_4 + m_5 + m_6 + m_7
 \end{aligned}$$



3. A traffic light controller receives a 4-bit input that changes every 5 seconds. The input sequence is a simple counter that counts from binary 0 to binary 15 and then starts again from binary 0. These signals go to 4 outputs, NSH, NSL, EWH, EWL. The first two outputs NSH and NSL respectively represent the high and low bits that are fed to the lamps that face in the North-south direction. The other two outputs EWH and EWL respectively represent the high and low bits that are fed to the lamps that face in the East-West direction. The following table indicates how setting the high and low bits determines the color of the lamp:

High	Low	Lamp Color
0	0	Green
0	1	Yellow
1	0	Red
1	1	Red

Each light should be green for 30 seconds, yellow for 5 seconds, and then red for 45 seconds. There should be two 5 second intervals when both lights are red. Assume that for the interval where the input is 0000, the North-South light has just turned green, and the East-West light has been red for 5 seconds already.

Design the circuitry that feeds from $ABCD$ to the four boolean outputs. You may represent your answer as algebraic equations (make sure to simplify).

Answer:

Time	Input	NS Color	NSH	NSL	EW Color	EWL	EWL
0	0000	Green	0	0	Red	1	X
1	0001	Green	0	0	Red	1	X
2	0010	Green	0	0	Red	1	X
3	0011	Green	0	0	Red	1	X
4	0100	Green	0	0	Red	1	X
5	0101	Green	0	0	Red	1	X
6	0110	Yellow	0	1	Red	1	X
7	0111	Red	1	X	Red	1	X
8	1000	Red	1	X	Green	0	0
9	1001	Red	1	X	Green	0	0
10	1010	Red	1	X	Green	0	0
11	1011	Red	1	X	Green	0	0
12	1100	Red	1	X	Green	0	0
13	1101	Red	1	X	Green	0	0
14	1110	Red	1	X	Yellow	0	1
15	1111	Red	1	X	Red	1	X

NSH:

AB	CD			
	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	1	1	1	1
10	1	1	1	1

NSL:

AB	CD			
	00	01	11	10
00	0	0	0	0
01	0	0	X	1
11	X	X	X	X
10	X	X	X	X

EWH:

AB	CD			
	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	0	0	1	0
10	0	0	0	0

EWL:

AB	CD			
	00	01	11	10
00	X	X	X	X
01	X	X	X	X
11	0	0	X	1
10	0	0	0	0

$$NSH = A + BCD$$

$$NSL = BC$$

$$EWH = \bar{A} + BCD$$

$$EWL = BC$$

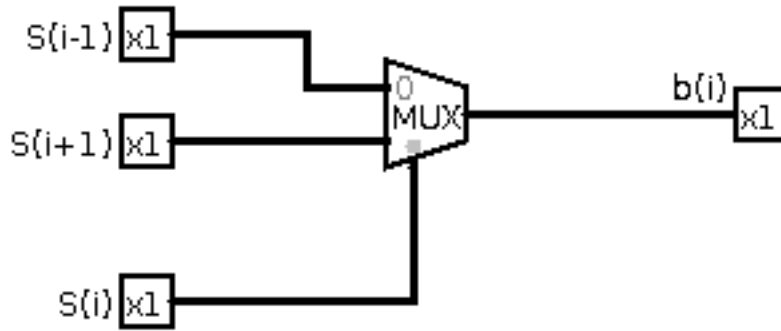
4. The 01-swap operation, b , on a binary string S , permutes all occurrences of 01 within the original string to 10 (the process is not recursive). For instance:

- $b(0) = 0, b(1) = 1, b(00) = 00, b(01) = 10, b(11) = 11$
- $b(000) = 000, b(001) = 010, b(010) = 100, b(100) = 100$
- $b(0\ 01\ 01\ 1\ 01\ 0) = 0\ 10\ 10\ 1\ 10\ 0$ (spacing added for clarity).

Design a circuit using 2-1 multiplexers that can be used to perform the 01-swap on a k -bit string $S = S_{k-1}S_{k-2} \cdots S_0$, where each S_i is a bit.

- (a) First, show the circuit, built using the 2:1 MUX, whose output is the i th bit of $b(S)$ where $0 < i < k - 1$. YOU DO NOT NEED ANY AND, OR, OR NOT GATES, only a single 2:1 MUX. This is somewhat challenging - so think what input information you need.

Answer: To compute the i th bit, feed S_i as the selector, S_{i+1} as high bit, S_{i-1} as low bit. i.e., if S_i is 0, swap S_{i-1} . This either changes $S_i = 0, S_{i-1} = 1$ to 10, or just swaps a 0 with 0. If $S_i = 1$, swaps $S_{i+1} = 0$ with $S_i = 1$ to 10, or just swaps 1 with 1.



- (b) Use contraction to solve the edge cases when $i = 0, k - 1$. You do not have to simplify the internals of the MUX, just explain why you “contracted” as you did.

Answer: For S_0 , feed in a '0' to MUX low, so if $S_0 = 0$, 0 gets swapped in. Note that this could be simplified to $S_{i-1}S_i$. For S_{k-1} , feed in a 1 in place of S_{i+1} , so that if $S_{k-1} = 1$, a 1 gets swapped in. Note that this can be simplified to $S_i + S_{i-1}$.

5. The isolated-1-shift-left operation (i1sl for short) applied to a binary string S moves a 1 bit by one position to the left in the solution if the bit is not adjacent to any other 1's (i.e., 0's on both sides, or if it is the least-significant bit, a 0 immediately above it).

For instance, the following show application of i1sl to various strings:

- $010 \rightarrow 100, 011 \rightarrow 011, 01010 \rightarrow 10100$
- $011010 \rightarrow 011100, 011100 \rightarrow 011100$

- (a) Build a simplified circuit (using only AND, OR, NOT gates) whose output is the i th bit of the i1sl, where $1 < i < k - 1$ (there's a hint here about what inputs are needed). You may leave your answer in algebraic (SoP) form in terms of the S_j . Note that the i th bit can be determined by only looking at a few of the S_j .

Answer: Do a truth-table / KMap for ith bit using i+1st, ith, i-1st, i-2nd bit. We need the i-2nd bit when ith, i-1st bits respectively are 0,1. Whether the "1" moves into ith position depends on i-2nd bit being a 0.

	$S_{i+1}S_iS_{i-1}S_{i-2}$	Out
0	0000	0
1	0001	0
2	0010	1
3	0011	0
4	0100	0
5	0101	0
6	0110	1
7	0111	1
8	1000	0
9	1001	0
10	1010	1
11	1011	0
12	1100	1
13	1101	1
14	1110	1
15	1111	1

Out(i):

		$S_{i-1}S_{i-2}$			
		00	01	11	10
$S_{i+1}S_i$	00	0	0	0	1
	01	0	0	1	1
	11	1	1	1	1
	10	0	0	0	1

$$\text{Out} = S_{i-1}\bar{S}_{i-2} + S_iS_{i-1} + S_{i+1}S_i$$

- (b) Suppose it is known that the input string will never contain 3 consecutive 1's. Draw the simplified circuit.

Answer:

	$S_{i+1}S_iS_{i-1}S_{i-2}$	Out
0	0000	0
1	0001	0
2	0010	1
3	0011	0
4	0100	0
5	0101	0
6	0110	1
7	0111	X
8	1000	0
9	1001	0
10	1010	1
11	1011	0
12	1100	1
13	1101	1
14	1110	X
15	1111	X

Out(i):

	$S_{i-1}S_{i-2}$			
$S_{i+1}S_i$	00	01	11	10
00	0	0	0	1
01	0	0	X	1
11	1	1	X	X
10	0	0	0	1

$$\text{Out} = S_{i-1}\bar{S}_{i-2} + S_{i+1}S_i$$

- (c) Suppose the input string might contain 3 1's, but that no consecutive 4 bits contain 3 0's (i.e., 0000, 0001, 0010, 0100, or 1000 never appear as a substring), nor does 0101 ever appear (1010 might still appear though, e.g., 111011011010). Draw the simplified circuit.

Answer:

	$S_{i+1}S_iS_{i-1}S_{i-2}$	Out
0	0000	X
1	0001	X
2	0010	X
3	0011	0
4	0100	X
5	0101	X
6	0110	1
7	0111	1
8	1000	X
9	1001	0
10	1010	1
11	1011	0
12	1100	1
13	1101	1
14	1110	1
15	1111	1

Out(i):

	$S_{i-1}S_{i-2}$			
$S_{i+1}S_i$	00	01	11	10
00	X	X	0	X
01	X	X	1	1
11	1	1	1	1
10	X	0	0	1

$$\text{Out} = \overline{S_{i-2}} + S_i$$

- (d) Use contraction to design the circuits for the outputs of the $k - 1$ st, 1st and 0th bits.

Answer: k-1 bit:

	$S_i S_{i-1} S_{i-2}$	Out
0	000	0
1	001	0
2	010	1
3	011	0
4	100	1
5	101	1
6	110	1
7	111	1

Out(i):

	$S_{i-1} S_{i-2}$	00	01	11	10
S_i					
0		0	0	0	1
1		1	1	1	1

$$\text{Out} = S_{i-1} \bar{S}_{i-2} + S_i$$

This can also be reached by fixing S_{i+1} to 1 and doing some boolean algebra.
1st bit:

	$S_{i+1} S_i S_{i-1}$	Out
0	000	0
1	001	1
2	010	0
3	011	1
4	100	0
5	101	1
6	110	1
7	111	1

Out(i):

	$S_i S_{i-1}$	00	01	11	10
S_{i+1}					
0		0	1	1	0
1		0	1	1	1

$$\text{Out} = S_{i-1} + S_{i+1} S_i$$

This can also be reached by fixing S_{i-2} to 0.
0th Bit:

	$S_{i+1} S_i$	Out_i
0	00	0
1	01	0
2	10	0
3	11	1

Out_i :

	S_i	0	1
S_{i+1}			
0		0	0
1		0	1

$$\text{Out} = S_{i+1} S_i$$

This can also be reached by fixing S_{i-2} and S_{i-1} to 0.

